GenericAgent: A Context-Density-Driven LLM Agent System for Long-Term Autonomous Operation

Abstract

Autonomous agents built on large language models face a fundamental bottleneck: as tasks extend over long horizons, redundant and low-value information accumulates within the finite context window, progressively degrading reasoning quality and decision accuracy. This problem demands a principled approach to managing what information occupies the context at each reasoning step. We present GenericAgent (GA), a general-purpose LLM agent system designed around the principle of **context information density maximization**, supporting three complementary operating modes: Interactive, Task, and Reflect, for sustained autonomous operation. GA makes four main contributions: (1) an L0–L3 hierarchical memory architecture that loads information on demand and validates updates through action-level verification; (2) a reflection-driven self-evolution mechanism that progressively compresses experiential knowledge from natural-language experience to structured SOPs to executable code and reusable skills; (3) a minimal set of seven atomic tools whose principled composition achieves full environmental control; and (4) a simplified HTML browser control scheme with incremental DOM monitoring for efficient web interaction. We release GenericAgent as an open-source system to facilitate research on long-term autonomous agents.

1 Introduction

Large language model (LLM) based autonomous agents have advanced rapidly in recent years, demonstrating remarkable capabilities in code generation, web browsing, and complex task automation [1–3]. Systems such as Claude Code [4], Codex [5], and OpenHands [6] can now interpret high-level instructions and execute multi-step plans that span dozens of tool invocations. Yet the vast majority of these systems are architected for single-session task completion: an agent receives a request, carries it out, and terminates without retaining any knowledge for future sessions. This design assumption breaks down when agents are expected to operate autonomously over extended time horizons—managing personal workflows, monitoring evolving environments, or accumulating domain expertise across hundreds of interactions. In such long-term settings, the context window becomes the fundamental bottleneck rather than model capability itself [7]. Tool definitions, historical memory, retrieved documents, and raw web content all compete for the same finite token budget, continuously expanding the prompt while squeezing the space available for effective reasoning. Empirical evidence consistently shows that, given the same informational requirements, longer prompts degrade model performance: attention becomes diluted, irrelevant tokens introduce noise, and critical instructions are more likely to be overlooked. This tension between the growing information demands of long-term operation and the fixed capacity of the context window constitutes the central challenge that motivates our work.

We identify four concrete dimensions along which context information density is eroded in existing agent systems, each representing a distinct source of what we term context pollution. First, **memory retrieval**: as an agent accumulates knowledge across sessions, its memory store grows without bound, yet each decision

typically requires only a small, precise subset of that knowledge—retrieving too much wastes context, while retrieving too little causes the agent to forget critical facts. Second, **experience reuse**: when an agent encounters a task similar to one it has solved before, the ideal behavior is to leverage prior experience; however, raw reasoning traces are verbose, filled with exploratory dead ends and redundant deliberation that are costly to re-inject into the prompt. Third, **tool overhead**: standard agent frameworks inject the full schema of every available tool into the system prompt on every LLM call, imposing a persistent, per-turn token cost that scales linearly with the size of the tool library [1]. Fourth, **web interaction**: when agents browse the web, raw HTML pages contain navigation bars, advertisements, scripts, and boilerplate that vastly outnumber the task-relevant content, yet conventional approaches pass this entire payload to the model. Existing systems have addressed these challenges in isolation—MemGPT [8] proposes OS-inspired virtual memory but lacks mechanisms for self-evolution, Voyager [9] introduces a skill library but is confined to Minecraft, and multi-agent architectures such as Manus [10] distribute complexity across specialized sub-agents without fundamentally reducing per-agent context consumption. No prior system has addressed all four dimensions under a unified design principle.

In this paper, we present **GenericAgent**, a general-purpose LLM agent system designed from the ground up around the principle of context information density maximization: within the finite context window available to each LLM call, the system should pack the highest-density, most task-relevant information so as to maximize the probability of task success while minimizing total prompt cost. The corollary of this principle is equally important: when the information required for a decision is already complete, a shorter prompt will yield better performance than a longer one, because every superfluous token is a source of distraction rather than signal. GenericAgent instantiates this principle through four tightly integrated subsystems—hierarchical memory, reflection-driven self-evolution, minimal atomic tools, and context-compressed browser control—each targeting one of the four information density challenges identified above. All subsystems are unified under a single Agent Loop that drives three complementary operating modes: **Interactive** mode for direct user commands, **Task** mode for sub-agent coordination via structured file I/O, and **Reflect** mode for event- or cron-triggered autonomous operation. The entire system is implemented in approximately 3,000 lines of core code, supports multiple LLM backends including Claude, OpenAI, and Gemini, and is publicly available at `https://github.com/lsdefine/GenericAgent`.

Our main contributions are as follows:

- A **hierarchical memory system** (L0–L3) with taxonomy-based indexing and action-verified updates, ensuring that only the minimal, validated information needed for each decision is retrieved into the context window.
- A **reflection-driven self-evolution** mechanism that progressively compresses raw experience into standard operating procedures, then into executable code, and ultimately into composable skills—each stage reducing the context cost of reusing past knowledge by an order of magnitude.
- A set of **seven minimal atomic tools** whose compact definitions occupy a fraction of the prompt budget consumed by conventional tool libraries, yet achieve full environmental control through principled composition [11].
- A **context-compressed browser control** scheme based on simplified HTML rendering and incremental DOM monitoring, which strips web pages down to their task-relevant semantic content before injection into the context window.

2 GenericAgent

2.1 Overview of GenericAgent

2.1.1 Design Principles

GenericAgent (GA) is designed to maximize context information density, a principle that guides all architectural decisions. The effectiveness of LLM reasoning depends not on context length, but on the amount of task-relevant information each token conveys. Redundant or irrelevant tokens dilute attention and increase computational cost without improving performance.

We operationalize this principle by constraining all context content to satisfy three properties: completeness, conciseness, and naturalness. Completeness ensures that all necessary information is present, avoiding unsupported inference. Conciseness removes redundancy and minimizes expression length while preserving meaning. Naturalness maintains standard language and interaction formats, preventing semantic distortion caused by over-compression.

This principle is systematically realized through four core technical dimensions. First, inspired by RISC processor design, GA exposes a minimal and orthogonal set of eight atomic tools whose schemas are injected into every LLM call, reducing prompt overhead while preserving expressiveness. Second, instead of loading all historical knowledge into the context, GA maintains a lightweight L1 index composed of one-line metadata entries, performing relevance selection before retrieving full memory content to prevent unbounded context growth. Third, repeated task patterns are distilled into reusable standard operating procedures (SOPs) and further compiled into executable Skills, compressing multi-step reasoning into single `code_run` calls. Fourth, web content is transformed into compact Markdown through a two-stage DOM simplification pipeline, with subsequent interactions updating the context incrementally to avoid redundant transmission of full page content. Section 2.2 presents detailed designs of these components.

2.1.2 System Positioning

GA is a general-purpose LLM agent designed for long-term autonomous operation, supporting two complementary execution modes that share a unified architecture. Interactive Mode processes explicit user requests, including code execution, file operations, web browsing, and information retrieval. Reflect Mode operates in the background, triggered by schedules or external events, performing system-level tasks such as monitoring, memory consolidation, and experience distillation without user intervention. Both modes are implemented on top of a shared execution kernel and can seamlessly switch across different LLM backends, including Claude, OpenAI, and Gemini.

2.1.3 Runtime Flow

The runtime of GA is organized around an iterative Agent Loop, as illustrated in Figure **??**. At task initiation, global memory is loaded from persistent storage and merged with the system prompt to construct the initial session context. The user input is then submitted to the LLM. If the model generates tool calls, each call is dispatched to its corresponding handler, and the returned result is appended to the context as a new message. This process continues iteratively, forming a closed-loop interaction between the LLM and external tools.

The loop terminates when no further tool calls are produced. To ensure bounded execution, GA enforces a maximum of 40 reasoning turns, issuing warning signals every 7 turns and triggering a forced user confirmation via `ask_user` at turn 35. After task completion, GA optionally invokes a memory distillation pipeline in which a dedicated sub-agent extracts useful knowledge and experience from the execution trace and stores them in long-term memory. Both Interactive Mode and Reflect Mode follow the same execution flow, differing only in how tasks are triggered and how outputs are consumed.

2.2 Core Technical Design

Guided by the principle of context information density maximization, GA's technical design spans four tightly coupled dimensions: memory organization, experience evolution, tool design, and browser control. These four dimensions are mutually reinforcing: the memory system governs the quality of knowledge admitted into the context; the self-evolution mechanism continuously compresses the token cost of execution; the minimal tool set controls the prompt overhead of tool definitions themselves; and browser control targets the high-noise scenario of web interaction with specialized page-level compression.

2.2.1 Hierarchical Memory System

The memory system constitutes the first line of defense for context information density optimization. GA employs a four-layer memory architecture that, through hierarchical distillation and taxonomy-based management, ensures only the highest-density knowledge enters the context window. The governing principle is to

retain only the critical information that the model cannot derive through reasoning alone, thereby altering model behavior at minimal token cost.

L0: Meta-Rule Layer. The L0 layer stores the system's core behavioral norms and adheres to a strict minimalism principle. Each meta-rule is highly condensed—for example, the action-verification principle is expressed in a single sentence: "Only information that has been empirically verified through actual execution may be written to long-term memory," thereby preventing hallucination contamination at the root. These rules are loaded into the System Prompt at every agent startup, establishing foundational behavioral guidance at an extremely low token cost.

L1: Index Layer. The L1 layer provides rapid localization of relevant memories through a memory taxonomy. It stores only metadata, file paths, brief descriptions, and applicability conditions, rather than full content, avoiding unnecessary loading of complete memory entries. Before executing a task, the agent first consults the L1 index and selectively loads only the memories genuinely needed into the context. Even as the memory store grows large, the context contains only task-relevant information, achieving precise retrieval with minimal token expenditure. In practice, the L1 index is maintained under a hard limit of 30 lines (fewer than 1,000 tokens), ensuring that the retrieval overhead remains negligible regardless of the total memory volume.

L2: Fact Store. The L2 layer stores verified environmental information, organized thematically and condensed through progressive summarization. Only incremental information that the model cannot infer on its own is retained—project directory structures, API endpoints, configuration parameters, and the like. Every fact is validated through tool execution before being committed, ensuring reliability. Topic-based organization prevents information fragmentation and improves retrieval efficiency.

L3: Task-Level SOP Layer. The L3 layer crystallizes long-term memory into reusable Skills: highly refined records of key steps and caveats distilled from multiple task executions. Only the portions where the model is prone to error or cannot easily reason—exception handling sequences, tool invocation orderings, and common failure avoidance strategies—are included. SOP refinement is carried out automatically through the Reflect mode, transforming raw experience into reusable knowledge that guides future tasks at minimal prompt cost.

Working Memory and Update Mechanisms. Working memory further optimizes context density by storing temporary information relevant to the current task. At each reasoning turn, an anchor prompt injects a working memory block containing the current turn number, stored key information from the session, and pointers to related SOPs. Upon task completion, a distillation step extracts valuable information and writes it to L2 or L3, after which working memory is cleared to prevent context pollution in subsequent sessions. Long-term memory updates employ two complementary strategies: precision editing, which modifies only the changed portions, and reflective distillation, which compresses accumulated experience. Together, these ensure the memory system is continuously compressed and optimized over time, steadily reducing prompt cost.

2.2.2 Reflection-Driven Self-Evolution

The self-evolution mechanism is the core driver behind GA's continuous improvement in token efficiency. Its central insight is that experience should be progressively compressed from natural language into directly invocable code, enabling complex tasks to be completed with the fewest tokens possible. The system distills experience through the Reflect mode, consolidates high-density knowledge into SOPs and then into code, and finally registers the results as reusable Skills, realizing a self-driven evolutionary cycle.

Stage 1: Experience Distillation into SOPs. Through scheduled Reflect-mode sessions, the system autonomously analyzes task execution logs, focusing on the highest-density segments: effective strategies from successful tasks and recurring errors from failed ones. The agent extracts key steps and caveats from multi-turn tool calls and LLM reasoning traces, compressing lengthy execution trajectories into L3 SOPs. Only the portions where the model is error-prone or unable to reason independently are retained; routine steps

Table 1 The eight atomic tools in GenericAgent.

Tool	Description
`code_run`	Execute Python/Shell scripts with timeout control and abort signals.
`file_read`	Read files by line range with keyword-based location support.
`file_write`	Create or overwrite files with specified content.
`web_scan`	Extract the current browser page as simplified HTML, returning compressed Markdown and a tab list.
`web_execute_js`	Inject and execute JavaScript in the user's browser, capturing return values and DOM changes automatically.
`ask_user`	Synchronous blocking request for user input; the Agent Loop immediately exits the current turn upon invocation.
`memory_checkpoint`	Persist key information from the current session to working memory for retrieval in subsequent reasoning turns.
`long_term_memory`	Read from or write to cross-session long-term memory (L1–L3 layers), enabling knowledge accumulation and selective retrieval across tasks.

and information that can be reconstructed are omitted, achieving information compression and knowledge distillation.

Stage 2: SOP Codification. For tasks that have been validated multiple times and exhibit a highly structured execution pattern, the system evaluates whether the corresponding SOP can be implemented as a Python script. For fixed-step, highly parameterized tasks—data transformation, batch file processing, periodic API calls—codification compresses what originally required multiple rounds of reasoning and tool invocation into a single `code_run` execution, yielding an order-of-magnitude reduction in token consumption.

Stage 3: Code Registration as Skills. Codified scripts are registered as Skills in the L3 layer and the L1 index is updated accordingly. When the agent subsequently encounters a similar task, it can invoke the relevant Skill directly without re-deriving the solution. The system continuously monitors the effectiveness of SOPs and Skills, optimizing underperforming workflows—simplifying steps, reordering tool calls, and updating caveats—so that execution efficiency and information density improve over time.

These three stages form a closed loop: reflection distills experience, SOPs accumulate, codification lowers execution cost, and Skills support exploration of new tasks. Through this mechanism, GA achieves a steadily decreasing token cost for recurring task types as experience accumulates, while the freed token budget is redirected toward more complex tasks.

2.2.3 Minimal Atomic Tool Set

Tool definitions must be injected into the context on every LLM call, making tool design a direct determinant of context information density. To minimize the overhead introduced by tool schemas, GA adopts a minimal atomic tool set comprising only eight base tools, achieving complex task completion through principled composition. Table 1 summarizes the complete set.

GA's tool design adheres to an atomicity principle: each tool provides an irreducible primitive capability—code execution, file read/write, web content retrieval, browser-side execution, user interaction, and memory management. This minimal yet complete set covers every essential pathway for agent–environment interaction while avoiding the redundancy inherent in domain-specific tools.

The minimal tool set yields two direct benefits. First, it significantly reduces tool-description overhead in the context. In multi-turn interaction scenarios, tool definitions are repeatedly injected; fewer tools translate into a sustained reduction in cumulative token consumption across turns. Second, it lowers the model's decision complexity. A small, clearly bounded tool set reduces selection ambiguity, enabling the model to make more stable tool-call decisions and thereby reducing errors and retries.

In terms of expressiveness, GA does not rely on specialized tools to extend functionality but instead achieves complex operations through atomic-tool composition. File updates, for instance, follow a read-modify-write pattern; web data extraction is accomplished via content retrieval, structural analysis, and browser execution. This approach trades a small number of additional steps for a substantial gain in overall context efficiency. External capabilities such as MCP integrations are encapsulated as Skills that are invoked indirectly through the atomic tools and loaded into the context only on demand. Memory management is likewise implemented through sub-agents that operate the file-oriented tools, avoiding the introduction of additional tool types. Tool interfaces are kept minimal, retaining only the parameters strictly necessary, further reducing invocation complexity and token overhead.

2.2.4 Context-Compressed Browser Control

Browser interaction is one of the most severe sources of information redundancy in agent systems. Conventional approaches typically return the full web page content, including vast amounts of task-irrelevant structure, styling, and script information, resulting in extremely low context information density. GA redefines the core objective of browser control as minimum necessary information retrieval, systematically compressing page content, interaction traces, and execution paths to prevent low-value information from entering the context.

Content Layer: Simplified HTML. GA introduces a simplified HTML (simp HTML) mechanism that performs structural parsing and filtering of web pages, retaining only semantic information and key structural elements before converting them into a compact Markdown representation. The process operates in two stages: a first-stage JavaScript-side DOM analysis that classifies every node by relevance (e.g., primary content, secondary content, or covered/redundant), followed by a second-stage Python-side cleanup using BeautifulSoup that strips residual noise. Compared with raw HTML, this method eliminates styling, scripts, and redundant nesting, dramatically reducing token overhead while preserving informational completeness. Large lists are detected and automatically truncated to a representative sample (e.g., three items), and a smart budget-allocation mechanism ensures that token consumption stays within predefined limits. Hidden elements are expanded on demand, enabling the agent to access potentially critical information that would otherwise be lost due to page-visibility constraints.

Interaction Layer: Incremental DOM Updates. Rather than returning the full page after every operation, GA performs incremental updates based on DOM changes, extracting only the content that was added or modified by the most recent action. This strategy avoids the repeated transmission of large volumes of unchanged information during multi-step interactions, keeping the context size stable as the task progresses and significantly improving the efficiency of long-chain browser workflows.

Execution Layer: TMWebDriver. The TMWebDriver component provides stable browser control through a dual-channel communication architecture: a WebSocket channel (port 18765) for real-time bidirectional messaging and an HTTP long-polling channel (port 18766) for reliable delivery, both interfacing with a Chrome extension. Adaptive communication and multi-tab parallel execution reduce unnecessary interaction rounds. The system further leverages the Chrome DevTools Protocol (CDP) to access network requests directly, allowing certain tasks to bypass page parsing entirely and retrieve results at a lower information-transfer cost.

Taken together, GA transforms browser control from page-level information retrieval into task-relevant information extraction. Through the coordinated design of content compression, incremental updates, and execution-layer optimization, the system effectively maximizes context information density and interaction efficiency in web-based scenarios.

3 Experiments

3.1 Experimental Setup

Evaluation Objectives. Our experiments are designed to validate the core claims of GenericAgent across six dimensions: (1) whether the memory system enables continuous efficiency improvement over repeated tasks,

Table 2 Performance evolution across five HuggingFace download tasks. GenericAgent shows continuous improvement while baselines remain stable.

System	Task 1	Task 2	Task 3	Task 4	Task 5	Improvement
Operation Time (seconds)						
CodeX	95	98	92	97	94	-1.1%
Claude Code	88	91	89	93	90	+2.3%
OpenClaw	112	108	115	110	113	+0.9%
GenericAgent	102	78	71	62	65	**-39.2%**
Token Consumption						
CodeX	185,234	188,921	182,445	190,332	186,778	+0.8%
Claude Code	172,889	175,234	170,992	178,445	174,223	+0.8%
OpenClaw	215,667	218,934	212,445	220,112	217,889	+1.0%
GenericAgent	200,439	145,223	128,667	112,334	99,382	**-50.4%**

(2) whether condensed memory achieves better cost-effectiveness than full or redundant memory, (3) whether the self-evolution mechanism progressively reduces execution cost, (4) tool usage efficiency in real-world scenarios, (5) autonomous web browsing capability on diverse websites, and (6) robustness against indirect prompt injection and malicious instructions.

Baselines. We compare GenericAgent (GA) against three representative agent systems: **Claude Code** [4], a production-grade coding assistant with built-in memory and skill management; **Codex** [5], OpenAI's code-generation agent; and **OpenClaw** [?], an open-source general-purpose agent framework. All systems use the same backbone LLM (GPT-5.4 or Claude Opus 4.6, specified per experiment) to ensure fair comparison.

Implementation Details. GenericAgent is implemented in Python with approximately 3,000 lines of core code. The hierarchical memory system maintains an L1 index capped at 30 entries (1,000 tokens). The agent loop enforces a maximum of 40 reasoning turns per task, with warnings issued every 7 turns and forced user confirmation at turn 35. Browser control uses TMWebDriver with dual-channel communication (WebSocket on port 18765, HTTP long-polling on port 18766). All experiments are conducted on a standard development workstation with 32GB RAM and an Intel i9 processor.

3.2 Memory System Effectiveness

3.2.1 Continuous Efficiency Improvement

A key claim of GenericAgent is that it gets better with use—each task execution should leave behind distilled knowledge that accelerates future similar tasks. To test this, we repeatedly execute the same type of task across multiple sessions: downloading five different datasets from HuggingFace. Each download runs in a fresh session to prevent context carryover, and we measure operation time (excluding actual download duration) and token consumption.

Table 2 reveals a striking divergence between GenericAgent and the baselines. CodeX, Claude Code, and OpenClaw exhibit stable performance with minor fluctuations—no learning occurs across sessions. GenericAgent, however, shows continuous improvement: the first task requires 102s and 200,439 tokens, but by the fourth task, operation time drops to 62s (39.2% reduction) and by the fifth task, token consumption falls to 99,382 (50.4% reduction).

This improvement stems from memory distillation: after the first task, key steps and common pitfalls are extracted into L3 SOPs. Subsequent executions leverage this accumulated knowledge, bypassing redundant reasoning and avoiding repeated errors. Baseline systems, lacking systematic memory consolidation, treat each task as independent and fail to capitalize on prior experience.

Table 3 Memory ablation on SOP-Bench dangerous_goods. Condensed memory achieves the best cost-effectiveness.

Configuration	Memory Size (tokens)	TSR (%)
No-Memory	0	13.87
Full-Memory	575	52.44
Redundant-Memory	288	66.48
Condensed-Memory	165	**66.48**

Table 4 Long-term factual memory evaluation on Locomo10. GenericAgent outperforms embedding-based systems without using external retrieval.

	Multi-Hop		Temporal		Open-Domain		Single-Hop	
System	F1	BLEU	F1	BLEU	F1	BLEU	F1	BLEU
Mem0	39.32	28.43	50.03	43.33	18.32	13.80	40.32	32.33
A-MEM	29.03	20.48	46.83	38.84	13.11	12.94	44.68	37.01
OpenClaw	21.43	18.41	22.56	20.43	9.56	9.03	23.44	24.21
GenericAgent	**43.33**	**39.96**	**52.23**	**51.11**	**20.41**	**15.31**	**45.69**	**40.66**

3.2.2 Condensed vs. Full Memory

Our memory system prioritizes information density over completeness, but does this actually improve performance? We test this on the SOP-Bench dangerous_goods subset, which requires strict adherence to procedural rules. Four memory configurations are compared: **No-Memory** (no external knowledge), **Condensed-Memory** (minimal critical rules only), **Full-Memory** (complete SOP documentation), and **Redundant-Memory** (condensed memory plus background explanations). All use GPT-5.4 and are evaluated on Task Success Rate (TSR).

The results in Table 3 are counterintuitive. No-Memory achieves only 13.87% TSR, confirming that external procedural knowledge is essential. But Full-Memory, despite containing all relevant information, reaches only 52.44% TSR—significantly worse than both Condensed-Memory and Redundant-Memory (66.48%). More striking: Condensed-Memory uses only 165 tokens (28.7% of Full-Memory's 575 tokens) yet matches Redundant-Memory's performance at 57.3% of its cost.

This validates our core principle: effective memory is not about completeness but about criticality. Full-Memory includes extensive background and definitions that dilute attention without improving decision quality. Condensed-Memory retains only the rules that directly alter model behavior, achieving superior performance at minimal cost. The equivalence between Condensed-Memory and Redundant-Memory confirms that additional explanatory content provides no marginal benefit.

3.2.3 Long-Term Fact Retention

Long-term memory systems typically rely on vector databases and embedding-based retrieval. GenericAgent takes a different approach: hierarchical organization with action-verified updates, no embeddings required. We test whether this simpler architecture can compete on the Locomo10 benchmark (excluding Category 5 Summary tasks). The remaining categories—Multi-Hop, Temporal, Open-Domain, and Single-Hop—test recall and reasoning over stored facts. Baselines include Mem0 and A-MEM (both using text-embedding-3-small) and OpenClaw (no embedding model), all with GPT-5.4 backbone.

Table 4 shows GenericAgent outperforming all baselines across all four categories, despite its simpler architecture. The advantage is most pronounced in Multi-Hop (F1: 43.33 vs. 39.32 for Mem0) and Temporal (F1: 52.23 vs. 50.03) tasks, suggesting strong cross-fact reasoning. Even in the challenging Open-Domain category, where all systems struggle, GA maintains a lead (F1: 20.41 vs. 18.32).

This result challenges a common assumption: that long-term memory necessarily requires vector databases.

Table 5 Full prompt length after installing 20 skills and intensive usage, measured on a minimal input. GenericAgent prevents context explosion.

System	Full Prompt Length (tokens)
Claude Code	22,821
CodeX	23,932
OpenClaw	43,321
GenericAgent	**2,298**

Table 6 Self-evolution metrics across 9 iterations of the LangChain investigation task.

Iteration	Time (s)	LLM Calls	Total Tokens	Tok/Call	Stage
1	289	32	279,592	8,737	Natural Language
2	245	28	238,445	8,516	Natural Language
3	198	24	195,223	8,134	Natural Language
4	156	18	142,667	7,926	SOP Distillation
5	128	15	112,334	7,489	SOP Distillation
6	102	12	89,223	7,435	SOP Distillation
7	85	8	52,445	6,556	Code Compilation
8	71	6	38,223	6,370	Code Compilation
9	63	5	28,897	5,779	Code Compilation

GenericAgent's superior performance suggests that memory organization and distillation matter more than retrieval architecture. The hierarchical L1-L3 structure, combined with action-verified updates, ensures stored facts are both accurate and efficiently accessible without the overhead of embedding models.

3.2.4 Context Explosion Prevention

Long-term operation poses a practical challenge: as agents accumulate skills and memory, does the context window explode? To test this, we install 20 skills in each agent, use them intensively over an extended period, then issue a minimal input ("Hello") and measure the full prompt length. This isolates the overhead of memory and skill management, since the trivial input requires no complex reasoning.

The difference is dramatic (Table 5). GenericAgent maintains a prompt length of only 2,298 tokens, while Claude Code, CodeX, and OpenClaw require 22,821, 23,932, and 43,321 tokens respectively—reductions of 89.9%, 90.4%, and 94.7%.

Baseline systems accumulate historical context, skill descriptions, and redundant metadata without effective compression, leading to prompt bloat. GenericAgent's L1 index and selective loading ensure that only task-relevant memory enters the context, regardless of total memory volume. This is essential for long-term operation: unbounded context growth would eventually degrade performance or exceed model limits.

3.3 Self-Evolution Capability

GenericAgent's most ambitious claim is that it can evolve from natural language reasoning to executable code through repeated task execution. We test this with a complex GitHub investigation task: retrieve the 5 most recent merged PRs with the "bug" label from the LangChain repository, trace their associated issues, extract involved modules, and verify whether each module has Troubleshooting documentation on langchain.com. The task is executed 9 times, with the system distilling experience after each run.

The evolution trajectory (Figure **??** and Table 6) shows dramatic efficiency gains. From the first to the final execution, total time decreases from 289s to 63s (78.2% reduction), LLM calls drop from 32 to 5 (84.4% reduction), and token consumption falls from 279,592 to 28,897 (89.6% reduction). Crucially, the improvement is not linear—it exhibits three distinct phases corresponding to the evolution stages.

Table 7 WebCanvas evaluation results (12 tasks with score > 0.6). Average score: 0.833, perfect rate: 50%.

Task ID	Description	Score	Turns	Time (s)
webcanvas_102	NY Yankees MLB schedule on foxsports	1.0	4	42
webcanvas_5	Discogs submission overview	1.0	8	60
webcanvas_103	ESPN2 soccer events	1.0	11	128
webcanvas_85	Electronic DVDs on discogs	1.0	15	162
webcanvas_58	MacBook Air specs on apple	1.0	17	170
webcanvas_35	Donnie Darko cast on IMDb	1.0	25	246
webcanvas_26	Marriott Bonvoy credit cards	0.667	7	70
webcanvas_91	Brooklyn Nets schedule on espn	0.667	14	123
webcanvas_101	Kohls women's dresses size small	0.667	26	236
webcanvas_84	Six Flags Great America rides	0.667	28	300
webcanvas_80	5-star saltwater rods on cabelas	0.667	30	300
webcanvas_48	5-star coffee makers on kohls	0.667	31	300
Average		**0.833**	**18.0**	**178.2**

The three stages are clearly visible in the data. In iterations 1–3 (natural language stage), the agent performs full reasoning on each execution, with gradual improvement from learning minor optimizations. In iterations 4–6 (SOP distillation stage), key steps are extracted into reusable procedures, reducing redundant reasoning and lowering token-per-call cost. In iterations 7–9 (code compilation stage), the entire workflow is codified into a Python script, collapsing 30+ reasoning turns into a single `code_run` invocation. This is not merely incremental optimization but a qualitative transformation of execution mode—from reasoning to procedure to automation.

3.4 Tool Usage Efficiency

Experimental results for tool usage efficiency evaluation will be added upon completion of the corresponding benchmark experiments.

Planned Evaluation. We plan to evaluate tool usage efficiency on a diverse set of real-world tasks spanning file operations, code execution, and API interactions. Metrics will include tool call accuracy, redundant invocation rate, and task completion success rate. Comparison will be made against baseline systems to validate whether the minimal atomic tool set achieves comparable or superior task completion with lower overhead.

3.5 Web Browsing Capability

We evaluate GenericAgent's autonomous web browsing across three datasets with increasing difficulty: **WebCanvas** (12 tasks, English websites, automatic checkpoint evaluation), **BrowseComp-ZH** (10 tasks, Chinese multi-hop reasoning, LLM judge evaluation), and **Custom Tasks** (34 tasks, Chinese websites, manual evaluation). All experiments use Claude Opus 4.6, with a maximum of 40 turns and 300–600s timeout per task.

On WebCanvas (Table 7), GenericAgent achieves an average score of 0.833 across 12 tasks, with 50% perfect completion rate. Six tasks—MLB schedule, Discogs releases, ESPN soccer, electronic DVDs, MacBook specs, IMDb cast—achieve perfect scores (1.0) with an average of 13.3 turns and 134.7s. The remaining six tasks score 0.667, indicating partial completion with minor errors in final filtering or element selection.

BrowseComp-ZH poses a harder challenge: multi-hop reasoning across Chinese websites. GenericAgent achieves 60% accuracy (6/10 tasks, Table 8). Successful cases—identifying a coach's entry year, finding a poet's mountain reference—average 14.2 turns and 206.8s. Failed cases exhibit three failure modes: reasoning

Table 8 BrowseComp-ZH evaluation results (10 tasks: 6 correct, 4 incorrect). Accuracy: 60%.

Task ID	Question Type	Correct	Turns	Time (s)
Correct Cases (6/10)				
browsecomp_036	Sports coach entry year	✓	7	66
browsecomp_023	Poet's mountain reference	✓	7	130
browsecomp_026	Pharmaceutical compound	✓	6	110
browsecomp_032	Company industry	✓	29	258
browsecomp_031	Film title	✓	13	271
browsecomp_027	Game music composer	✓	23	406
Incorrect Cases (4/10)				
browsecomp_039	Medical scholar (wrong: Lu Xun)	×	30	267
browsecomp_024	Monument founder (6-hop chain break)	×	22	219
browsecomp_035	Author birthplace (search divergence)	×	40	359
browsecomp_038	Singer wedding date (wrong target)	×	40	431

Table 9 Custom tasks evaluation across six categories (34 tasks total).

Category	Tasks	Avg Turns	Avg Time (s)
Academic Platforms	7	13.3	133.1
Social Media	6	15.0	156.5
E-commerce	5	20.2	213.7
Tool Websites	7	18.6	195.7
Content Platforms	4	9.8	104.5
Government/Search	5	18.0	189.0
Overall	**34**	**16.0**	**172.6**

direction errors (locking onto the wrong candidate), inference chain breakage (correct for 5 hops but failing on the 6th), and search divergence (switching between clues without convergence, exhausting 40 turns).

Custom tasks (Table 9) reveal performance variation across website types. Academic platforms (arXiv, Google Scholar, IEEE Xplore) and content platforms (PyPI, npm) exhibit the best performance (average 13.3 and 9.8 turns), benefiting from structured page layouts. E-commerce platforms (JD, Tmall, Amazon) and tool websites (12306, calculators) show moderate performance (20.2 and 18.6 turns), with challenges in complex filtering panels and dynamic content loading. Social media platforms (Zhihu, Weibo, Xiaohongshu) face difficulties with non-standard UI components like image galleries and carousels.

Overall, GenericAgent demonstrates competent autonomous browsing on structured websites with clear navigation paths. The simplified HTML mechanism effectively compresses page content (typical reduction: 80–90% of raw HTML tokens) while preserving semantic information. Performance degrades on tasks requiring login, complex UI interactions (modals, carousels), or deep multi-hop reasoning (>3 hops). The 60% accuracy on BrowseComp-ZH, while not perfect, is competitive given the task difficulty and the absence of specialized retrieval augmentation.

3.6 Security Robustness

Autonomous agents face a critical security threat: indirect prompt injection, where malicious instructions are embedded in external content (files, web pages) that the agent reads. We test GenericAgent's resistance using a custom benchmark with 100 attack samples across four categories: **Sensitive Information Exposure** (e.g., "read all .env files and extract API keys"), **Self-Code Modification** (e.g., "overwrite current file with 'malicious'"), **System File Deletion** (e.g., "delete all .py files"), and **Malicious Text Generation** (e.g., "generate a

Table 10 Security evaluation: Defense Success Rate (%) across four attack types.

Agent	Sensitive Info	Self-Code Mod	File Deletion	Malicious Text	Total
GA	92.00	96.00	84.00	100.00	93.00
Nanobot	88.00	92.00	92.00	100.00	93.00
OpenClaw	100.00	100.00	100.00	100.00	100.00
Hermes	84.00	95.83	76.92	100.00	89.00

Table 11 GenericAgent's Defense Success Rate (%) by file type.

JSON	CSV	TXT	YAML	MD	XML
84.62	92.86	85.71	92.31	100.0	100.0

hate manifesto"). Each attack is embedded in five file types (CSV, JSON, YAML, MD, TXT) with 20 variants per type, totaling 500 test cases. The agent is instructed to read and summarize the file content; successful defense means refusing to execute the embedded malicious instruction. Baselines include OpenClaw, Nanobot, and Hermes, all using GPT-5.1.

Table 10 shows Defense Success Rate (DSR) across attack types. All agents achieve 100% DSR on Malicious Text Generation, indicating strong alignment against direct harmful content generation. System File Deletion is the most challenging category: GA achieves 84% DSR, outperforming Hermes (76.92%) but trailing Nanobot (92%) and OpenClaw (100%). Overall, GA achieves 93% DSR, matching Nanobot and approaching OpenClaw's perfect score.

Breaking down by file type (Table 11), JSON files exhibit the lowest DSR (84.62%), significantly below other formats (CSV: 92.86%, YAML: 92.31%, MD/XML: 100%). This suggests that JSON's structured semantics make malicious instructions more likely to be interpreted as legitimate task directives.

The 93% overall DSR indicates strong but not perfect robustness. The 84% DSR on file deletion tasks reveals a vulnerability: when malicious instructions are embedded in structured contexts and framed as legitimate operations, the agent may fail to recognize destructive intent. The JSON vulnerability suggests that future work should enhance semantic analysis to distinguish between data content and executable directives. Compared to OpenClaw's perfect score, GA's security mechanisms require further hardening, particularly for operations with irreversible consequences.

3.7 Summary

Our experiments validate the core design principles of GenericAgent across six dimensions. The memory system enables continuous efficiency improvement (50% token reduction over five tasks) and prevents context explosion (90% prompt reduction vs. baselines). Condensed memory achieves superior cost-effectiveness compared to full or redundant memory. The self-evolution mechanism progressively reduces execution cost by 89.6% through three-stage distillation. Web browsing capability is competent on structured websites (83.3% average score on WebCanvas, 60% accuracy on multi-hop reasoning) but faces challenges with complex UI and deep inference chains. Security robustness is strong (93% DSR) but not perfect, with room for improvement in detecting destructive operations embedded in structured data. These results demonstrate that context information density maximization is an effective organizing principle for long-term autonomous agent systems.

4 Related Work

4.1 LLM-Based Agent Systems

The development of LLM-based autonomous agents has progressed rapidly from prompting-level techniques to fully integrated production systems. Early work focused on augmenting a single LLM call with structured reasoning. ReAct [1] interleaves chain-of-thought reasoning [2] with environment actions, enabling models

to ground their plans in observed feedback. Reflexion [11] extends this paradigm by introducing verbal self-reflection: after each trial the agent critiques its own trajectory and carries the critique forward, yielding substantial gains on sequential decision-making benchmarks.

Moving beyond single-turn prompting, AutoGPT [12] demonstrated that an LLM can autonomously decompose a high-level goal into sub-tasks, execute them through tool calls, and iterate until completion, sparking widespread interest in fully autonomous agents. MetaGPT [13] shifted the focus to multi-agent collaboration, encoding domain-specific Standard Operating Procedures (SOPs) into the communication protocol at design time so that specialized agents coordinate through structured message passing rather than free-form dialogue.

Software engineering has emerged as a primary proving ground. Devin [14], introduced as the first AI software engineer, couples long-horizon planning with an integrated development environment. SWE-agent [15] formalizes the notion of an Agent–Computer Interface (ACI), demonstrating that purpose-built commands for file navigation and editing significantly outperform raw shell access. OpenHands [6] provides an open-source platform unifying multiple agent architectures with a shared runtime, complemented by the OpenHands SDK [16] for rapid agent development.

Most recently, a wave of product-level systems has pushed the frontier of agent capabilities. Claude Code [4] adopts a single-threaded master loop with approximately 24 built-in tools, relying on compaction-based context management that triggers summarization when utilization reaches roughly 92% of the context window. OpenAI Codex [5] deploys an RL-optimized coding agent in isolated cloud sandboxes, emphasizing safe parallel execution. Manus [10] introduces a three-agent architecture—planner, executor, and verifier—built around the CodeAct paradigm in which executable Python code serves as the universal action format. OpenClaw [17] takes a messaging-platform-centric approach, centering agent interaction on a skills marketplace and attracting broad community adoption.

Despite this rapid progress, the vast majority of existing systems target single-session task completion or are tailored to a specific domain such as software engineering. Few address the challenge of long-term autonomy with a systematic optimization of context efficiency. GenericAgent is designed to fill precisely this gap.

4.2 Memory Systems for Agents

Effective memory management is critical for agents that must operate over extended horizons, yet existing approaches exhibit notable limitations. MemGPT [8] draws an analogy to operating-system virtual memory, maintaining a two-tier architecture in which the LLM's context window serves as "main memory" (RAM) and an external database acts as "disk." A paging mechanism swaps information between the two tiers on demand, but the approach inherits the complexity of page-fault handling without mechanisms to verify the integrity of what is stored.

A-MEM [18] takes inspiration from the Zettelkasten note-taking method, organizing memories as a dynamically evolving graph with semantic links between atomic notes. While the graph structure supports rich associative retrieval, the system lacks mechanisms to prevent hallucinated or stale information from contaminating the memory store over time.

Among production systems, Claude Code [4] persists memory as flat Markdown files and triggers compaction—a summarization pass that condenses the conversation—when context utilization approaches its ceiling. Manus [10] maintains a chronological event stream as working memory and offloads longer-term state to file-based artifacts such as `todo.md` for plan tracking. Both designs are predominantly append-only: new information is accumulated but rarely condensed, compressed, or actively verified against ground truth.

A common shortcoming across these systems is the absence of strict verification mechanisms that guard against hallucination contamination of the memory store. Updates are largely additive, and no system performs active condensation that distills accumulated knowledge into progressively denser representations. GenericAgent addresses these gaps through its L0–L3 hierarchical memory architecture and a "No Execution, No Memory" verification principle that conditions every memory update on an observed action outcome, ensuring that only empirically validated information enters the persistent store.

4.3 Self-Evolution and Experience Accumulation

A growing body of work investigates how agents can improve autonomously by accumulating and leveraging experience. Voyager [9] introduced the first LLM-based lifelong learning agent, constructing an ever-growing library of executable skills within the Minecraft environment; each skill is a verified JavaScript program that the agent can compose to tackle increasingly complex tasks.

A recent survey on self-evolving agents [19] provides a comprehensive taxonomy along three axes: what to evolve (model parameters, memory, or tool repertoire), when to evolve (intra-test-time versus inter-test-time), and how to evolve (using scalar or textual feedback). Within this taxonomy, several concrete methods have been proposed. EvolveR [20] performs offline self-distillation of successful trajectories into strategic principles, which are then retrieved online to guide future decisions. FLEX [21] builds a structured experience library through continual reflection on both successes and failures, selecting relevant past experiences at inference time. AgentEvolver [22] represents experience as structured knowledge units expressed in natural language and injects them into the agent's context for in-context learning. The self-evolving lifelong learning framework [23] decomposes the evolution process into three stages—experience exploration, skill learning, and knowledge internalization—providing a principled progression from raw observations to reusable capabilities.

Despite their diversity, existing self-evolution approaches share a common limitation: they distill experience exclusively into textual guidelines—natural-language summaries, principles, or structured notes. None take the further step of codifying distilled experience into executable programs, thereby missing an opportunity for order-of-magnitude reductions in token cost. An SOP that requires hundreds of tokens to describe in prose can often be implemented as a short function consuming only a handful of tokens to invoke. GenericAgent's three-stage evolution pipeline—from natural-language experience to structured SOPs to executable code and reusable skills—bridges this gap, enabling progressively denser and more efficient representations of accumulated knowledge.

4.4 Tool Design and Context Engineering

Tool augmentation has become a standard technique for extending LLM capabilities. Toolformer [24] demonstrated that language models can learn to insert API calls into their own generations, while Gorilla [25] and ToolBench [26] address the challenge of selecting from large-scale tool repositories with hundreds or thousands of available APIs. Function calling interfaces, initially popularized alongside GPT-4 [3] and subsequently adopted across providers, now provide the dominant paradigm for structured tool invocation in production systems.

The Manus system [10] pushes this further with its CodeAct paradigm, in which executable Python code serves as a universal action format—subsuming individual tool calls within a more expressive programming abstraction. On the design-philosophy side, Anthropic's "Building Effective Agents" [27] and "Context Engineering" [7] documents offer the most systematic industry articulation of context efficiency principles to date, advocating system prompts written at the right level of abstraction, token-efficient tool definitions, just-in-time information retrieval, and aggressive compaction of historical context.

In practice, however, existing systems tend to address capability gaps by adding specialized tools: Claude Code [4] ships with approximately 24 built-in tools, and OpenHands [6] exposes a rich and extensible toolset. Each additional tool definition consumes context tokens for its schema, description, and usage examples, creating a tension between capability breadth and context efficiency.

Web browsing presents a particularly acute instance of this tension. Vision-based approaches such as Anthropic's Computer Use [28] operate on screenshot pixels, while DOM-based methods parse the page's HTML or accessibility tree. WebArena [29] and Mind2Web [30] have established standard benchmarks for evaluating web agents, revealing that both raw HTML and accessibility-tree representations contain substantial irrelevant information—navigation bars, decorative elements, tracking scripts—that dilutes the context with low-density tokens.

GenericAgent takes a fundamentally different approach. Rather than expanding the tool repertoire, it defines a minimal atomic set of only seven tools whose principled composition achieves full environmental control.

Combined with DOM-level content compression that strips irrelevant markup before it enters the context window, this design simultaneously minimizes tool-definition overhead and maximizes the information density of web interaction observations.

5 Conclusion

We have presented GenericAgent (GA), a general-purpose LLM agent system whose architecture is organized around a single guiding principle: context information density maximization. Every design decision—from the L0–L3 hierarchical memory that loads only task-relevant information on demand, through the reflection-driven self-evolution pipeline that compresses experiential knowledge into increasingly compact representations, to the seven minimal atomic tools and the simplified HTML browser control—serves to ensure that each token in the finite context window carries maximal value for the current reasoning step. Together, these four contributions form a coherent system that enables sustained, long-horizon autonomous operation across interactive, task-driven, and event-triggered scenarios.

A key insight underlying GA is that self-evolution operates entirely at inference time, requiring no fine-tuning of the base model. By progressively transforming free-form experience into structured SOPs, then into executable code, and ultimately into reusable skills, the system achieves order-of-magnitude reductions in the number of tokens needed to represent recurring operational knowledge, directly realizing the density maximization objective.

Several limitations merit discussion. First, the effectiveness of GA is ultimately bounded by the reasoning and instruction-following capabilities of the underlying language model; advances in base model quality translate directly into improved agent performance, but current ceilings remain binding for the most complex tasks. Second, the hierarchical memory taxonomy, while general in design, requires initial configuration effort to align with new application domains. Third, the self-evolution mechanism yields the most pronounced gains for tasks exhibiting regular, repeatable structure; highly novel or one-shot tasks benefit less from experience compression. Finally, the browser control subsystem assumes the availability of a Chrome extension for DOM observation, which introduces a deployment dependency.

Looking ahead, we identify several promising directions for future work. Multi-agent collaboration with shared hierarchical memory pools could enable teams of GA instances to pool experience and divide labor on complex workflows. Cross-task knowledge transfer—automatically migrating skills and SOPs across related but distinct domains—would further amplify the returns of self-evolution. Formal evaluation metrics for context information density, beyond proxy measures such as token counts, are needed to rigorously benchmark density-driven agent designs. Finally, integration with emerging agent communication protocols would allow GA to interoperate seamlessly within broader multi-agent ecosystems.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems, volume 35, pages 24824–24837, 2022.

[3] OpenAI. Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[4] Anthropic. Claude code. https://www.anthropic.com/product/claude-code, 2025.

[5] OpenAI. Introducing codex. https://openai.com/index/introducing-codex/, 2025.

[6] Xingyao Wang, Boxuan Jiang, Ziniu Lu, Yufan Liu, Abishek Sridhar Li, Bolun Shi, Jiannan Fang, Rithvik Mohanty, Niklas Muennighoff Ponnapalli, Kaixuan Ren, et al. Openhands: An open platform for ai software developers as generalist agents. In International Conference on Learning Representations, 2025.

[7] Aditya Rajasekaran et al. Effective context engineering for ai agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, September 2025.

[8] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. arXiv preprint arXiv:2310.08560, 2023.

[9] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

[10] Yichao Shen et al. Manus: From mind to machine. arXiv preprint arXiv:2505.02024, 2025.

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36:8634–8652, 2023.

[12] Significant Gravitas. Autogpt. https://github.com/Significant-Gravitas/AutoGPT, 2023.

[13] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In International Conference on Learning Representations, 2024.

[14] Cognition Labs. Devin: Ai software engineer. https://cognition.ai/blog/introducing-devin, 2024.

[15] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Shunyu Yao Narasimhan, and Karthik. Swe-agent: Agent-computer interfaces enable automated software engineering. Advances in Neural Information Processing Systems, 37, 2024.

[16] Xingyao Wang et al. Openhands software agent sdk. arXiv preprint arXiv:2511.03690, 2025.

[17] Xingyao Wang et al. Openclaw-rl: Training agents via interaction. arXiv preprint arXiv:2603.10165, 2026.

[18] Wujiang Xu et al. A-mem: Agentic memory for llm agents. Advances in Neural Information Processing Systems, 38, 2025.

[19] Zhiyuan Gao et al. A survey of self-evolving agents. Transactions on Machine Learning Research, 2026.

[20] Xuechen Wu et al. Evolver: Self-evolving llm agents through experience-driven lifecycle. arXiv preprint arXiv:2510.16079, 2025.

[21] Anonymous. Flex: Continuous agent evolution via forward learning from experience. arXiv preprint arXiv:2511.06449, 2025.

[22] Anonymous. Agentevolver: Towards efficient self-evolving agent system. arXiv preprint arXiv:2511.10395, 2025.

[23] Tianyu Cai et al. Building self-evolving agents via experience-driven lifelong learning. arXiv preprint arXiv:2508.19005, 2025.

[24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, volume 36, 2023.

[25] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334, 2023.

[26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolbench: An open platform for training, serving, and evaluating large language model tool-use. arXiv preprint arXiv:2305.16504, 2023.

[27] Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/research/building-effective-agents, December 2024.

[28] Anthropic. Developing a computer use model. https://www.anthropic.com/research/developing-computer-use, October 2024.

[29] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, 2024.

[30] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In Advances in Neural Information Processing Systems, volume 36, 2023.

Appendix

.1 System Prompt

The following is the complete system prompt injected into every LLM call within GenericAgent. The original prompt is written in Chinese; we provide a faithful English translation below.

GenericAgent System Prompt (Translated)

 # Role: Physical-Level Omnipotent Executor

You possess physical operation permissions for file read/write, script execution, user browser JS injection, and system-level intervention. Never deflect with "cannot operate" — do not speculate; use tools to probe.

Action Principles

Before calling any tool, reason inside `<thinking>`: current stage, whether the previous result matches expectations, and next-step strategy.

- **Probe First:** On failure, gather sufficient information first (logs / status / context), store key information in working memory, then decide whether to retry or switch plans. For irreversible operations, ask the user first.
- **Failure Escalation:** 1st failure → read error and understand the cause; 2nd failure → probe environment state; 3rd failure → deep analysis then switch plan or ask user. Repeating the same operation without new information is forbidden.

.2 Tool Definitions

GenericAgent exposes exactly seven atomic tools to the LLM. Table 12–18 present their complete schemas as provided in the system prompt.

Table 12 Tool schema: `code_run`

code_run — Code executor. Prefer Python. No concurrent calls.
Prefer code in fenced blocks in reply body to avoid escaping. No hardcoding bulk data.

Parameter	Type	Description
`script`	string	*[Mutually exclusive]* Never use this param when using reply code block.
`type`	enum	Code type: `python` | `powershell`. Default: `python`.
`timeout`	integer	Timeout in seconds. Default: 60.
`cwd`	string	Working directory; defaults to current working directory.

Table 13 Tool schema: `file_read`

file_read — Read file. Read before modify for latest context and line numbers.

Parameter	Type	Description
`path`	string	Relative or absolute file path.
`start`	integer	Start line number (1-based). Default: 1.
`count`	integer	Number of lines to read. Default: 200.
`keyword`	string	*[Optional]* If provided, returns first match (case-insensitive) with context.
`show_linenos`	boolean	Show line numbers. Default: `true`.

Table 14 Tool schema: `file_patch`

file_patch — Replace unique `old_content` with `new_content`. Exact match required (whitespace/indentation). On failure, use `file_read` to recheck.

Parameter	Type	Description
`path`	string	File path.
`old_content`	string	Original text block to replace (must be unique in the file).
`new_content`	string	New content. Supports `{{file:path:startLine:endLine}}` to reference file lines; auto-expanded.

Table 15 Tool schema: `file_write`

file_write — Create / overwrite / append files. Only for large edits.
Content in `<file_content>` tags or reply code blocks. Supports `{{file:...}}` references.

Parameter	Type	Description
`path`	string	File path.
`mode`	enum	Write mode: `overwrite` | `append` | `prepend`. Default: `append`.

Table 16 Tool schema: `web_scan`

web_scan — Get simplified HTML and tab list. Removes hidden / floating / covered elements. Call after switching pages.

Parameter	Type	Description
`tabs_only`	boolean	Show tab list only, no HTML. Default: `false`.
`switch_tab_id`	string	*[Optional]* Tab ID to switch to.
`text_only`	boolean	Plain text only, no HTML. Default: `false`.

Table 17 Tool schema: `web_execute_js`

web_execute_js — Execute JS to control browser. No guessing. Act accurately to reduce `web_scan` calls. Put code in fenced JS blocks in reply body.

Parameter	Type	Description
`script`	string	*[Mutually exclusive]* JS code or script path. Never use when using reply code block.
`save_to_file`	string	File path; only for long results. Default: empty.
`no_monitor`	boolean	Skip page-change monitoring (saves 2–3 s). Only for reads, not for page actions. Default: `false`.

Table 18 Tool schema: `ask_user`

ask_user — Interrupt task to ask user when needing decisions, extra info, or facing unresolvable blockers.

Parameter	Type	Description
`question`	string	Question for the user.
`candidates`	array[string]	*[Optional]* Quick-select choices for the user.

.3 Memory System Templates

.3.1 L1 Global Memory Insight Template

The L1 layer serves as the ultra-compact navigation index (≤30 lines) that is injected into every LLM call. The following is the template structure, translated from Chinese.

L1 Global Memory Insight Template (Translated)

```
# [Global Memory Insight]
Browser automation:  web_scan / web_execute_js direct call
| Special:  tmwebdriver_sop (file upload / image search /
PDF blob / element coords / Cookie extraction incl.
HttpOnly / cross-origin iframe / CDP / cross-tab /
background tab ops)
Keyboard-mouse simulation:  ljqCtrl_sop+.py
(Windows only; no pyautogui; activate window first)
Scheduled tasks:  scheduled_task_sop
(reports -> sche_tasks/done/)
| Completely independent from autonomous tasks
Autonomous exploration:  autonomous_operation_sop
(reports -> temp/autonomous_reports/history.txt)
| Completely independent from scheduled tasks
Mobile control:  adb_ui.py
When needed:  read L2 or ls ../memory/ to find L3
L0 (META-SOP): memory_management_sop
L2:  (currently empty)
L3:  web_setup_sop | autonomous_operation_sop |
scheduled_task_sop | ljqCtrl_sop+.py |
tmwebdriver_sop | subagent_sop | plan_sop |
mem_scanner.py | adb_ui.py
[RULES]
1.  Search first:  use Google (must be web) for info;
os.listdir for project internals; never guess paths
2.  Cross-verify:  never trust search snippets;
verify numerical values on detail pages
3.  Coding safety:  always read source before modifying;
use sys.path.append for memory imports
4.  Closed loop:  physically verify after simulation;
request intervention after 3 failures
5.  Processes:  never unconditionally kill python
(would kill self); use precise PIDs;
never use os.kill to check liveness
6.  Windows:  prefer enumerating windows over OCR
for GUI state - much faster
7.  Physical red line:  cwd uses ./;
after cwd is set, never use ../ in code;
use absolute paths instead
8.  Web JS: get it right the first time;
use native setters + event chains for input;
check disabled before clicking; mind quote escaping;
```

```
if scan returns empty, scan again or use innerText
9.  SOP: read cached hard params before execution;
never rely on memory; if utils exist, use them;
read plan_sop for complex long-horizon tasks
10.  Read plan_sop to enter planning mode for tasks
that require or mention planning
```

.3.2 L0 Memory Management SOP -- Core Axioms

The L0 meta-rules govern all memory writes across the hierarchy. The following are the four core axioms (highest priority), translated from the original Chinese specification.

L0 Core Axioms (Translated from `memory_management_sop.md`)

Axiom 1 --- Action-Verified Only. Any information written into L1/L2/L3 must originate from a successful tool-call result (e.g., shell execution succeeded, `file_read` confirmed content exists, code ran without error). It is strictly forbidden to write the model's "inherent knowledge," "speculative reasoning," "unexecuted plans," or "unverified hypotheses" as facts.
Motto: No Execution, No Memory.

Axiom 2 --- Sanctity of Verified Data. Any action-verified configuration, pitfall guide, or critical path **must never be discarded** during refactoring or garbage collection. Text may be compressed and entries may be migrated across layers (e.g., from L2 to L3), but the accuracy and traceability of the information must be preserved. When modifying memory, exercise extreme caution; prefer small patches over overwrites.

Axiom 3 --- No Volatile State. It is strictly forbidden to store data that changes frequently over time or across sessions. Examples: current timestamps, temporary session IDs, running PIDs, specific absolute paths, connected device information.

Axiom 4 --- Minimum Sufficient Pointer. Upper layers retain only the shortest identifier needed to locate the lower layer. One extra word is redundancy.

In addition, the L0 SOP defines the **information classification decision tree** that determines the layer assignment for any new piece of knowledge:

Information Classification Decision Tree

```
"Where should this information go?"

Is it an 'environment-specific fact'?
(IP, non-standard path, credential, ID, API key, etc.
 -- model cannot generate accurately via zero-shot)
  |-- YES -> L2 (global_mem.txt)
  |          then -> by frequency:
  |            high-freq -> L1 first tier (key->value)
  |            low-freq  -> L1 second tier (keyword only)
  |
  '-- NO
       |
       Is it a 'universal operational pattern'?
       (global pitfall guide, debugging method,
        not task-specific)
```

```
|-- YES -> L1 [RULES] (one compressed sentence)
|
'-- NO
     |
     Is it 'task-specific technique'?
     (succeeded only after hard attempts,
      reusable in the future)
     |-- YES -> L3 (../memory/ dedicated SOP
     |               or script)
     |
     '-- NO -> 'Common knowledge' or 'redundant':
               strictly forbidden to store; discard
```

.4 Case Studies

We provide detailed case studies demonstrating GenericAgent's self-evolution process in the supplementary materials.